KubeQuest

Documentation Projet

Attendus du sujet & Realisation complete

Groupe	group-09
Ecole	Epitech Nice
Region AWS	Paris eu-west-3
Kubernetes	v1.29.15
Repository	T-CLO-902-NCE_7
Date	Juin 2026

1. Presentation du Projet

KubeQuest est un projet Epitech qui consiste a deployer une infrastructure Kubernetes complete et industrialisable sur AWS. L'objectif est de mettre en place toutes les briques d'une plateforme de production moderne : cluster, monitoring, logging, securite, GitOps et deploiement d'application.

> 💡 Le projet a ete realise de zero sur 4 instances EC2 AWS dans la region Paris (eu-west-3), en partant d'Amazon Linux 2023 vierge. Toute l'infrastructure est versionnee dans un depot Git et deployable en une seule commande.

1.1 Infrastructure AWS

Instance	IP Publique	Role	Services installes
kube-1 (node-1)	`13.36.145.234`	Control Plane	API Server, etcd, scheduler — IP Elastique fixe
kube-2 (node-2)	`35.180.92.133`	Worker + Ingress	nginx-ingress, pods app, pods monitoring
node-3	`13.39.82.213`	Reserve	Non utilise activement
node-4	`35.180.98.210`	Reserve	Non utilise activement

2. Tableau de Conformite — Attendus vs Realise

Ce tableau compare chaque exigence du sujet avec ce qui a ete realise.

Composant	Attendu par le sujet	Statut	Ce qui a ete fait
Cluster Kubernetes	*kubeadm, nodes*	✅ **Fait**	kube-1 (control plane) + kube-2 (worker) — v1.29.15 — Flannel CNI
Load Balancer Ingress	*nginx-ingress*	✅ **Fait**	DaemonSet sur kube-2 — hostNetwork=true — URLs nip.io publiques
Dashboard	*kubernetes-dashboard*	✅ **Fait**	v2.7.0 — ingress configure — accessible via token admin
Monitoring	*kube-prometheus*	✅ **Fait**	Prometheus + Grafana + AlertManager — namespace monitoring
Logging	*loki*	✅ **Fait**	Loki + Promtail — logs centralises — datasource Grafana
GitOps infra	*kustomize*	✅ **Fait**	gitops/overlays/production — ArgoCD synchronise automatiquement
Helm chart app	*helm*	✅ **Fait**	charts/sample-app/ — templates custom — MySQL StatefulSet
GitOps app	*kustomize*	✅ **Fait**	Synchronise par ArgoCD — Healthy + Synced
Automatisation deploy	*deploy-all.sh*	✅ **Fait**	bash deploy-all.sh — deploiement complet en une commande
Resource limits	*CPU/RAM limits*	✅ **Fait**	Definis sur tous les pods app — OPA force le respect

Secrets Kubernetes	*secretKeyRef*	✅ **Fait**	MySQL credentials injectes via Secret — plus de mots de passe en clair
Labels ressources	*app.kubernetes.io/**	✅ **Fait**	Labels normalises sur tous les objets applicatifs
Replicas + affinite	*plusieurs replicas*	✅ **Fait**	2 replicas minimum — PodAntiAffinity — HPA 2-5
Stockage persistant	*PersistentVolume*	✅ **Fait**	PVC 5Gi pour MySQL — local-path-provisioner
Backup BDD	*CronJob*	✅ **Fait**	CronJob quotidien 2h — dump MySQL — PVC dedie — testable en live
OPA Gatekeeper	*webhook validation*	✅ **Fait**	Policies : resource limits, app-name label, images sans tag
Dex + oauth2-proxy	*authentification*	✅ **Fait**	OIDC — namespace security — protege Grafana, Dashboard, ArgoCD
ArgoCD (bonus)	*GitOps avance*	✅ **Fait**	Bonus — Healthy+Synced — sync auto depuis GitHub
nip.io DNS (bonus)	*URLs publiques*	✅ **Fait**	Bonus — 6 services accessibles publiquement sans DNS custom
Auto-scaling demo	*HPA demo*	✅ **Fait**	251% CPU → scale 2→5 replicas — valide en live
Rollback demo	*broken deploy*	✅ **Fait**	Image cassee → detection → helm rollback → retour stable en 30s

3. Detail de la Realisation

3.1 Cluster Kubernetes

Installation manuelle avec kubeadm sur Amazon Linux 2023 (aarch64). Containerd comme moteur de conteneurs avec SystemdCgroup=true. Flannel pour le reseau inter-pods (CIDR 10.244.0.0/16).

```
# Initialisation sur kube-1
sudo kubeadm init --pod-network-cidr=10.244.0.0/16
--apiserver-advertise-address=10.0.9.232

# Jonction de kube-2
sudo kubeadm join 10.0.9.232:6443 --token <token>
--discovery-token-ca-cert-hash sha256:<hash>

# Verification
kubectl get nodes
```

```
# ip-10-0-9-232   Ready   control-plane   v1.29.15
# ip-10-0-9-137   Ready   <none>          v1.29.15
```

3.2 Ingress & URLs publiques

nginx-ingress deploye via Helm sur kube-2. 6 services exposes publiquement via nip.io (pas de DNS custom necessaire).

Service	URL
Application Laravel	`http://app.35.180.92.133.nip.io`
Grafana	`http://grafana.35.180.92.133.nip.io`
Kubernetes Dashboard	`http://dashboard.35.180.92.133.nip.io`
ArgoCD	`https://argocd.35.180.92.133.nip.io`
Dex	`http://dex.35.180.92.133.nip.io`
OAuth2 Proxy	`http://oauth2.35.180.92.133.nip.io`

⚠️ L'IP de kube-2 change a chaque redemarrage. Mettre a jour les ingress avec la nouvelle IP via kubectl patch ingress.

3.3 Stack Monitoring

kube-prometheus-stack installe Prometheus, Grafana et AlertManager en un seul chart Helm. Loki + Promtail pour la centralisation des logs. Correction appliquee : suppression de isDefault=true sur la datasource Loki pour eviter le conflit avec Prometheus.

- Prometheus : collecte metriques CPU, RAM, HTTP, latence
- Grafana : dashboards pre-configures — admin / prom-operator
- AlertManager : gestion des alertes avec seuils configurables
- Loki : centralisation des logs de tous les pods
- Promtail : agent DaemonSet sur chaque noeud

3.4 Application Laravel + MySQL

Application PHP 8.2 + Apache deployee via Helm chart custom. Image buildee et pushee sur Docker Hub (hanenetriaa/sample-app:latest).

- 2 replicas minimum avec PodAntiAffinity
- HPA : 2 a 5 replicas selon charge CPU (seuil 70%)
- MySQL StatefulSet avec PVC 5Gi (local-path-provisioner)
- CronJob backup MySQL tous les jours a 2h du matin
- Secrets injectes via secretKeyRef — aucun mot de passe en clair
- PDB pour garantir la disponibilite pendant les maintenances

3.5 Securite

Deux niveaux de securite complementaires.

OPA Gatekeeper

Webhook de validation qui intercepte chaque creation/modification de ressource. Policies actives :

- require-resource-limits : tout pod doit avoir CPU et RAM limits
- require-app-name-label : label app.kubernetes.io/name obligatoire
- deny-latest-tag : images sans tag specifique interdites

Dex + OAuth2 Proxy

Dex fournit l'identite OIDC. OAuth2 Proxy protege les applications. Compte admin : admin@kubequest.local.

- Probleme resolu : DNS dex.kubequest.local ajoute dans CoreDNS
- Probleme resolu : issuer passe de HTTPS a HTTP (pas de certificat SSL)
- cookieSecret genere dynamiquement avec openssl

3.6 GitOps avec ArgoCD

ArgoCD surveille le depot GitHub et synchronise automatiquement le cluster. Application configuree sur gitops/overlays/production.

✅ Status : Healthy + Synced — Last sync : quelques secondes apres le dernier commit

3.7 Demos validees en live

✅ HPA : charge 2000 requetes → CPU 251%/70% → scale automatique 2 → 5 replicas

✅ Rollback : image cassee deployee → detection echec → helm rollback → retour stable en 30 secondes

✅ Backup : kubectl create job --from=cronjob → job complete en 9 secondes

✅ ArgoCD : push sur GitHub → synchronisation automatique < 1 minute

4. Points Techniques Importants

4.1 Problemes resolus

Probleme	Solution
Image Bitnami MySQL introuvable	Chart custom avec mysql:8.0 officiel + local-path-provisioner
Grafana CrashLoopBackOff	Suppression isDefault=true sur datasource Loki (conflit)
DNS dex.kubequest.local inexistant	Ajout hosts block dans CoreDNS ConfigMap
Dex issuer HTTPS sans certificat	Config issuer en HTTP (http://dex.kubequest.local:5556)
OPA bloque ArgoCD (pas de limits)	ArgoCD installe avant OPA — warnings non bloquants
nodeSelector bloque Dex	--set nodeSelector=null lors de l'installation Helm
cookieSecret format invalide	openssl rand -base64 24 | tr -d newline/equal
Dashboard bloque HTTP	Ajout --enable-insecure-login dans les args du deployment
nginx-ingress sur mauvais noeud	hostNetwork=true — ingress colle au noeud kube-2

4.2 Limites connues

- Single control plane : si kube-1 tombe, plus de nouveaux deployments (apps continuent)
- IPs dynamiques : kube-2/3/4 changent a chaque redemarrage, mettre a jour les ingress
- Pas de TLS : certificats auto-signes — Let's Encrypt (cert-manager) en roadmap
- Dex en HTTP : pas de chiffrement sur le flux d'authentification